

# Optimized Piecewise Affine Abstractions of Neural Networks with Learnable Activation Functions

Noah Schwartz<sup>\*†</sup>, Chandra Kanth Nagesh<sup>\*†</sup> , Sriram Sankaranarayanan<sup>†</sup> 

Ramneet Kaur<sup>‡</sup>, Tuhin Sahai<sup>‡</sup>, Susmit Jha<sup>§</sup>

<sup>†</sup> University of Colorado Boulder, USA. Email: first.lastname@colorado.edu

<sup>‡</sup>SRI International. Email: first.lastname@sri.com

<sup>§</sup>DARPA. Email: first.lastname@darpa.mil

**Abstract**—We present a generalized framework for the range verification of neural networks featuring non-linear activation functions. Our approach first constructs an “optimized piecewise affine abstraction” of the network that replaces each non-linear activation function by a piecewise affine (PWA) function plus a bounded error. Such PWA functions are readily amenable to existing neural network verification techniques using specializations of linear arithmetic SMT solvers and mixed-integer optimization approaches. However, there are infinitely many ways to abstract each node, with a natural tradeoff between the number of pieces used, the global error bound, and the complexity of the resulting verification problem. We propose a dynamic programming (DP) algorithm to systematically compute the optimized PWA abstraction for general activation functions, guaranteeing tighter output bounds. The algorithm combines a local DP approximation at each node with a global error bound, yielding a variant of the knapsack problem for deciding how to allocate a fixed budget on the total number of pieces across units so as to minimize the worst-case error bound between the network and its approximation. Although the knapsack problem is itself NP-hard, we can use pseudo-polynomial DP algorithms as well as approximation schemes to solve it efficiently. Crucially, our approach is broadly applicable to diverse networks consisting of non-linear activations, including standard Multi-Layer Perceptrons (MLPs) and recently proposed architectures such as Kolmogorov-Arnold Networks (KANs). **On KAN benchmarks spanning 60 to 1,010 parameters, our approach yields output bounds that are significantly tighter than uniform PWA allocation, with the gap widening as networks grow. Furthermore, as the networks grow larger, the overhead of performing an abstraction pays off in terms of the computation time for solving each verification problem.**

## I. INTRODUCTION

We examine piecewise affine (PWA) abstractions for solving range verification problems for feedforward neural networks whose nodes (neurons) feature a wide class of nonlinear and “learnable” activation functions. Range verification involves computing bounds on the outputs of the network, given a set of inputs [1], [2]. Range verification serves as a primitive for verifying properties of neural networks for applications such as control of safety-critical autonomous systems [3], [4] and robustness of networks to input perturbations [5], [6]. Commonly used neural networks typically employ activation functions such as sigmoid, ReLU, tanh, Leaky ReLU and GeLU [7]. These functions are fixed as part of the network architecture and typically do not change as the network is trained. Recently proposed network models such as *deep spline networks* [8], [9] and Kolmogorov-Arnold Networks [10], [11]

employ “free-form” activation functions modeled using B-splines or radial basis functions which can change during training time in order to better fit the data. As a result, such networks have highly nonlinear activation functions that are not fixed *a priori*. At the same time, commonly used approaches for solving neural network verification problems have relied on the activation functions of the nodes in the network being piecewise affine (PWA) functions [4], [5] or being abstracted by a PWA function with known error bounds. However, the strategy for approximating activation functions by PWA functions is often fixed *a priori* without consideration of the specific neural network instance.

In this paper, we observe that the choice of a PWA abstraction is, in itself, a decision problem: (a) how many pieces do we use to approximate a given unit in the NN? (b) how does our choice of pieces affect the overall error between the function represented by the NN and its PWA approximation? and (c) how do we optimally choose the number of pieces at each unit locally so that the global error estimate can be minimized? Our framework proposed in this paper provides answers to these questions through dynamic programming.

The key steps of our approach are illustrated in Fig. 1. Let  $f_N : \mathbb{R}^n \rightarrow \mathbb{R}$  be the function defined by a NN. We assume single output for the ease of presentation. We construct a PWA approximation  $\hat{f}_N : \mathbb{R}^n \rightarrow \mathbb{R}$  as follows:

- 1) We consider a set of possible choices for approximating each unit activation function  $\psi$  in the network by PWA functions  $\psi_1, \dots, \psi_P$  with increasing number of pieces and decreasing error. We adapt a classic DP algorithm first proposed by Bellman and Roth for this purpose [12], [13]. We construct a “trade-off” table for each unit based on the error  $e_j = \max_z |\psi(z) - \psi_j(z)|$ , as the number of pieces  $j$  varies.
- 2) *Error Analysis*: We bound the worst case error  $e_{\max} = \max_{x \in X} |f_N - \hat{f}_N|$  between the NN and its PWA approximation as a function of the errors made at each individual unit. Theorem 1 of the paper shows that this error bound can be written as a weighted linear combination of the error bounds obtained for each unit.
- 3) *Knapsack Formulation*: We find a surprising connection between the problem of finding an “optimized” abstraction and solving a variant of the *knapsack* problem: a combinatorial optimization problem that involves the optimal

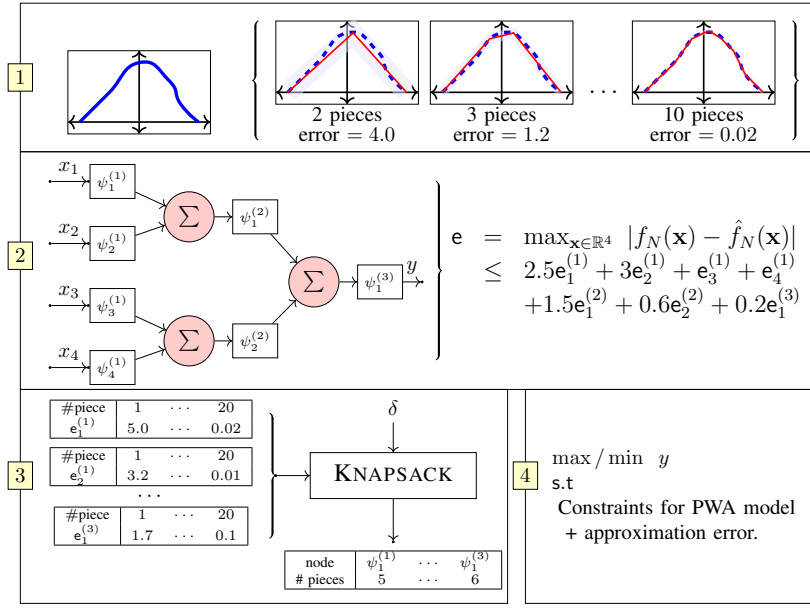


Fig. 1: An illustration of the key steps of our approach. 1. We consider multiple possible choices for each unit/activation function; 2. We upper bound the overall network-wide approximation error as a linear combination of individual errors; 3. We solve a knapsack problem to choose an optimized approximation that minimizes the error bound; and 4. We connect our PWA abstraction to existing NN verification solvers, especially the mixed integer approach.

choice of items that maximize value while constraining total weights to fall below a budget.

- 4) *Connecting Back to Known Activation Functions:* Having identified an optimized allocation of the number of pieces and error at each node of the network, we prove that any PWA function can be written as a weighted combination of ReLU units (Theorem 4). This allows us to connect our PWA abstraction to a host of existing NN verification techniques that specialize to ReLU network.
- 5) *MILP Formulation:* We solve the range verification problem, using Mixed Integer Linear Programming (MILP), while encoding the error bounds for the PWA approximation in the MILP model. This allows the verification problem to be translated into a MILP, which can be solved through powerful combinatorial optimization solvers such as Gurobi [14], CPLEX [15] and MOSEK [16]. In fact, by translating a PWA function into a weighted sum of ReLU units, our approach may in fact integrate well within existing NN verification tools.

We focus our empirical evaluation on a recently proposed class of feedforward networks with learnable activation functions called Kolmogorov Arnold Networks (KANs) [10], [11] that have become popular across many applications in scientific machine learning. Our experiments on KANs trained for four tasks of increasing scale demonstrates that the upfront cost of the knapsack-based abstraction is recovered many times over during verification. **Our optimized abstraction yields output bounds that are up to  $22\times$  tighter than uniform PWA allocation, with the largest gap under small-perturbations where verification is most valuable. The advantage grows with both**

**the size of the KAN and the width of the input bounds.** However, we note that the overhead of computing the optimized abstraction can be significant, especially for smaller networks, when compared to the time taken to solve the overall range verification problem. That said, the abstraction can be solved once and reused for multiple verification instances. This is often the case for applications such as certifying robustness of neural networks against adversarial perturbations wherein the certification is performed over ranges that arise from a large set of test examples [17]. Also, the use of the optimized abstraction shows gains in the computation time for the verification problem in many instances even when the abstraction overhead is taken into account.

## II. RELATED WORK

### A. Verification Problems and Their Significance

This paper focuses on verification problems for neural networks and, in particular, the range analysis problem. Neural networks are now used in safety-critical applications such as self-driving cars [18], robotics [19], and smart prosthetics [20]. In this context, they suffer from some unique challenges beyond incorrect decisions arising from lack of training data and overfitting. Adversarial attacks were first reported by Goodfellow et al. [21], where small input perturbations on datasets like MNIST-10 (handwritten digits) led to wrongful classifications with high probability. Since then, several techniques have been developed to train models to prevent such attacks. A detailed survey can be found here [22], [23].

Verification tools have addressed these issues by proving that a given neural network is robust to certain types of perturbations

or synthesizing a counterexample that shows a vulnerability in the network. Existing tools have been focused on feedforward neural networks. They include Reluplex [4], MARABOU (SMT-based) [24], CROWN (uses convex relaxations) [25], SHERLOCK (MILP-based) [26] and others [27] (see [28], [1], [2] for a detailed survey). These tools reduce the complex verification question to range verification, the precise problem being tackled in this paper. They have been used to certify safety of an aircraft collision avoidance system [3], [4], prove bounds on inputs to control systems [29], study the sensitivity of cyber-physical models to perturbations [30] and certify robustness to certain types of adversarial attacks [5], [31].

### B. NNs with Learnable Activation Functions

Neural networks with learnable “free form” activation functions such as deep-spline networks have been shown to be advantageous over those with fixed activation functions [9], [8] in terms of being able to learn more succinct and accurate function representations but at the cost of increased training time. More recently, Kolmogorov Arnold Networks (KANs) also replace fixed nonlinear activation functions with general univariate functions that can change during the training process. This leads to networks that are more parameter-efficient than MLPs, while still maintaining competitive or superior performance [11]. KANs have been shown to achieve better accuracies when compared with larger MLPs for the same learning task [32], [33]. Applications include time-series predictions [34], vision [35] and physics-based learning [36]. Due to the nonlinear and free-form nature of these activation functions, these networks are not amenable to existing range verification approaches beyond those based on simple interval bound propagation [37], which suffers from the well-known wrapping effect and consequently tends to yield very coarse bounds on networks of this kind.

### C. Neural Network Abstraction

Abstracting a neural network with activation functions such as tanh, sigmoid and gelu by means of a PWA function has been investigated as part of most range verification approaches [2]. Pulina and Tacchella were one of the first to propose a form of piecewise affine abstractions by replacing each nonlinear activation function by a series of intervals [38]. Their abstraction of nonlinear activation functions subdivides the domain uniformly using a single width parameter  $p$  and uses the fact that commonly used activation functions (sigmoid and tanh) are monotonically increasing functions to bound the value of the function in each subinterval. If a given large value of the width parameter is unable to prove the property, the refinement process lowers the width produce a finer grained abstraction. In contrast, our work here uses a PWA approximation that is optimized in two senses: we use Bellman’s algorithm at each node to find the best way to grid the domain into  $k$  pieces that minimizes the approximation error [13], [12], and we use different number of pieces at different nodes. While refinement is not part of the approach in this paper, we can implement a refinement loop that is similar in spirit to the work of Pulina

and Tacchella by incrementally increasing the total number of pieces used in our abstraction.

Following Pulina and Tacchella, most subsequent approaches either focus exclusively on networks with piecewise affine activation functions such as ReLU [27], [4], [39], [40], or employ an *a priori* fixed abstraction that is performed once for each activation function to achieve a desired local error bound (see for example [26]). Accounting for this error yields a sound output range that can be used to establish properties. While such approaches can work for activation functions such as sigmoid and tanh that are well approximated by PWA functions with up to 5 pieces, they fail for networks that employ B-splines or radial basis functions as their activation functions.

Bound propagation approaches such as CROWN [25] or DeepPoly [31] can eliminate the need for an upfront abstraction by generating upper and lower bounds for nonlinear activation functions on-the-fly as the analysis is performed. At the same time, such approaches can yield coarse bounds if the input bounds are large and the activation function is not monotone. The recent work of Shi et al. [41] improves the handling nonlinear activation functions such as tanh, sinh and variants of ReLU using a generalized branch-and-bound approach that iteratively branches nonlinear activation functions at carefully chosen points, and uses the bound propagation approach of alpha-beta-CROWN [42]. Shi et al. assumes a pre-optimized set of branching points for their approach for a fixed library of commonly occurring nonlinear functions and chooses a single branching point on the fly, whereas our approach computes PWA approximation upfront for a given error tolerance  $\delta$  through dynamic programming. In a sense, the contributions of our work can likely improve the approach of Shi et al. by providing a principled approach for choosing branching points.

The work of Ivanov et al. uses a creative approach that encodes neural networks as nonlinear differential equations and converts neural network verification problems to hybrid systems verification problems [43]. While activation functions such as tanh and sigmoid can indeed be seen as satisfying a differential equation, it is not clear how arbitrary activation functions can be encoded in this framework.

### D. Verification for KANs and B-Spline Networks

To the best of our knowledge, few publications have explored the verification of KANs, and the field remains relatively unexplored. The related problem of robustness of KANs to adversarial perturbation has been investigated: Heiderich et al. [44] present a comparative analysis of KANs and MLPs in the context of adversarial robustness, concluding that standard KAN architectures are similarly vulnerable and introducing a new training method. [45] modify the KAN architecture by using Cubic Hermite splines in place of B-Splines to guarantee monotonicity: the range analysis problem for such networks is solved simply by evaluating the network at two corner-points of the input hyper-rectangle. Additional efforts include the works of [46] and [47], where they similarly highlight the importance of adversarial robustness and study techniques to improve the effectiveness of KANs against added noise. Our approach is

the first to consider the problem of systematically computing PWA abstractions of KANs using a reduction to a variant of the knapsack problem.

### III. PRELIMINARIES

In this section, we will formally define the neural network model and provide some key assumptions on the activation functions so that they can be approximated by a PWA function with finitely many pieces.

*Notation:* We will represent vectors  $\mathbf{x}, \mathbf{y}, \mathbf{z}$  using boldface notation. For a vector  $\mathbf{z}$  its  $j^{\text{th}}$  component will be written  $z_j$ . For vectors  $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ , we define  $\mathbf{z} = \mathbf{x} \odot \mathbf{y}$  (the Hadamard product) of two vectors as the  $n$  dimensional vector such that  $z_i = x_i y_i$ .

**Definition 1.** A general feedforward neural network with  $K \geq 0$  hidden layers with  $n$  inputs and  $m$  outputs is defined by

- 1) Weight matrices  $W_1, \dots, W_K, W_{K+1}$ , wherein each  $W_i$  has dimensions  $n_i \times n_{i-1}$  with  $n_0 = n$  and  $n_{K+1} = m$ , and
- 2) Layerwise activation functions  $\sigma_1, \dots, \sigma_K$  wherein  $\sigma_i : \mathbb{R}^{n_i} \rightarrow \mathbb{R}^{n_i}$  defines the nonlinear transformation at each hidden layer, and is itself a vector of  $n_i$  individual unit-level activation functions  $\psi_{i,j} : \mathbb{R} \rightarrow \mathbb{R}$ .

$$\sigma(\mathbf{z}) = \begin{pmatrix} \psi_{i,1}(z_1) \\ \vdots \\ \psi_{i,n_i}(z_{n_i}) \end{pmatrix}$$

The function computed by the network is given by

$$f_N(\mathbf{x}) = W_{K+1} \times \sigma_K(W_K \times \dots \times \sigma_2(W_2 \times \sigma_1(W_1 \times \mathbf{x}))).$$

**Assumption 1** (Neural Network Input Assumption). We assume that the inputs to the neural network fall inside a set  $\mathbf{x} \in X$  wherein  $X = [-M_1, M_1] \times \dots \times [-M_n, M_n]$  for fixed (but possibly large) bounds  $M_1, \dots, M_n \geq 0$ .

**Assumption 2** (Assumptions on  $\psi$ ). We will assume that each unit level activation function  $\psi : \mathbb{R} \rightarrow \mathbb{R}$  satisfies the following:

- 1)  $\psi(z)$  is continuous and differentiable.
- 2) There is a procedure `DERIVATIVEINTERVAL`( $\psi, a, b$ ) that given input interval  $z \in [a, b]$ , outputs an interval  $[\ell, u]$  such that  $[\ell, u] \supseteq \left\{ \frac{d\psi}{dz} \mid z \in [a, b] \right\}$ . Also, there exists an error tolerance  $\epsilon > 0$  such that: a)  $\ell \leq \min_{z \in [a, b]} \frac{d\psi}{dz} \leq \ell + \epsilon$ , b)  $u - \epsilon \leq \max_{z \in [a, b]} \frac{d\psi}{dz} \leq u$ . The `DERIVATIVEINTERVAL` procedure will help us sample derivatives of  $\psi$  at discrete points and reason about how the function behaves between these discrete points.

**Lemma 1.** As a consequence of Assumptions 1 and 2, for each unit  $\psi_{i,j}$  there exists a constant  $L_{i,j} \geq 0$  such that if the input to the network is  $\mathbf{x} \in X$ , then the input to the unit lies in the interval  $z \in [-L_{i,j}, L_{i,j}]$ .

*Proof.* Proof is a simple consequence of the continuity of each activation function which guarantees that the image of a compact (closed and bounded) set remains compact.  $\square$

In practice, the domain  $[-L_{i,j}, L_{i,j}]$  for each unit in the network can be over-approximated by a standard interval analysis carried out over the entire network [48].

These assumptions are satisfied for commonly used activation functions including gelu, tanh, sigmoid as well as learnable activation functions including B-Splines [11], [49], Padé approximations [50], Chebyshev polynomials [51], Gaussian processes [52], and Fourier Series [53].

**Definition 2** (Continuous PWA Function). A  $k$ -piece univariate, continuous PWA function (for  $k \geq 2$ ) is represented by a sequence of  $k-1$  breakpoints:  $\langle (z_1, y_1), \dots, (z_{k-1}, y_{k-1}) \rangle$  and end slopes  $m_0, m_{k-1}$ . The breakpoints satisfy the ordering constraint wherein  $z_1 < \dots < z_{k-1}$  such that, the PWA function  $\hat{\psi}$  is defined as:

$$\hat{\psi}(z) = \begin{cases} y_i + (z - z_i) \frac{y_{i+1} - y_i}{z_{i+1} - z_i} & \text{if } z \in [z_i, z_{i+1}] \\ y_1 + m_0(z - z_1) & z \leq z_1 \\ y_{k-1} + m_{k-1}(z - z_{k-1}) & z \geq z_{k-1} \end{cases}$$

where,  $i \in \{1, \dots, k-2\}$ . Let  $m_i := \frac{y_{i+1} - y_i}{z_{i+1} - z_i}$  for  $i \in \{1, \dots, k-2\}$ .

a) *Range Analysis Problem* : The range analysis problem seeks a guaranteed range for the outputs of a network given a set of inputs. Let  $f_N : \mathbb{R}^n \rightarrow \mathbb{R}$  be a NN with  $K$  layers as defined in Def. 1. Suppose we define a range  $\mathbf{x} \in [\ell, \mathbf{u}] \subseteq X$ , wherein  $\ell, \mathbf{u} \in \mathbb{R}^n$  are vectors and  $\mathbf{x} \in [\ell, \mathbf{u}]$  denotes the box constraint  $\ell_i \leq x_i \leq u_i$  for each component  $i \in \{1, \dots, n\}$ . Our goal is to find a “tight” range that over-approximates all the possible output values. In other words, we wish to find  $\alpha, \beta \in \mathbb{R}$  with  $\alpha \leq \beta$  such that

$$\forall \mathbf{x} \in [\ell, \mathbf{u}], f_N(\mathbf{x}) \in [\alpha, \beta] \quad (1)$$

Although  $\alpha = -\infty$  and  $\beta = \infty$  is a trivial solution, we often seek to find the tightest possible bounds that establish that  $[\alpha, \beta] \subseteq [a, b]$  for some user-provided correctness property

$$\mathbf{x} \in [\ell, \mathbf{u}] \Rightarrow f_N(\mathbf{x}) \in [a, b].$$

**Problem 1** (Range Verification Problem). Given a KAN  $N$  computing a function  $f_N : \mathbb{R}^n \rightarrow \mathbb{R}$ , an input range  $\mathbf{x} \in [\ell, \mathbf{u}]$ , we wish to find the “tightest-possible” range over the output  $[\alpha, \beta]$  satisfying Eqs. (1).

We can extend Problem 1 to cases wherein the inputs  $\mathbf{x}$  range over a polyhedron  $P[\mathbf{x}]$  since our approach in this paper is based on a MILP formulation and the polyhedral constraint over the inputs can be directly encoded.

### IV. OPTIMIZED PIECEWISE AFFINE (PWA) APPROXIMATIONS FOR UNIVARIATE FUNCTIONS

Let us consider a univariate function  $\psi(z) = \psi_{i,j}(z)$ , which appears as part of the neural network from Def. 1, satisfying Assumption 2. We will provide an approach to approximate it using a PWA function with a fixed bound on the number of pieces within the interval  $z \in [-L, L]$  wherein  $L = L_{i,j}$  corresponds to the input domain of this particular unit when the network input domain is the set  $X$  (see Assumption 1 and Lemma 1).

a) *Domain Discretization*: Since the function  $\psi$  is not known to us, we will discretize the continuous range  $z \in [-L, L]$  into  $j_{\max} + 1$  intervals using grid points that are  $\Delta = \frac{2L}{j_{\max}}$  distance apart, so that each grid point is of the form  $z_j = -L + j\Delta$  for a natural number  $j \in \mathbb{N}$ . Here,  $j_{\max}$  is a user-specified input, fixed once  $L$  and the discretization factor  $\Delta$  are chosen (equivalently,  $j_{\max} = 2L/\Delta$ ): a larger  $j_{\max}$  (smaller  $\Delta$ ) yields a finer grid and hence a smaller discretization error, at the cost of a longer running time for the dynamic program. We evaluate the function at each of these grid points and approximate the function over this finite set of discrete grid points. Later, we will show how to account for the error introduced by the interval between the grid points that is not accounted for in this process. The discretized optimal PWA problem is stated as follows:

**Problem 2** (Optimal Univariate PWA Approximation). *The problem of optimally approximating  $\psi(z)$  is as follows:*

**Inputs:** An algorithm for  $\psi$ , limits  $-L, L$ , discretization factor  $\Delta$  and limit on number of pieces  $k$ .

**Output:** Breakpoints  $z_1 \leq z_2 \leq \dots \leq z_{k-1}$ , which together with the values  $y_i = \psi(z_i)$  define (via Def. 2) a  $k$ -piece PWA function  $\hat{\psi}$ , such that the discretized error  $e(\Delta) = \max_{u \in \{-L, -L+\Delta, -L+2\Delta, \dots, L\}} |\psi(u) - \hat{\psi}(u)|$  is minimized.

Note that if two breakpoints  $z_j, z_{j+1}$  coincide in the output, we treat them as the single breakpoint.

Let  $n(j; \Delta) = -L + j\Delta$ . We write  $n(j) := n(j; \Delta)$  whenever  $\Delta$  is clear from context, so  $n(j)$  and  $n(j; \Delta)$  denote the same grid point. The optimal univariate PWA approximation can be solved using a dynamic programming algorithm first proposed by Bellman et al. [13], [12]. The algorithm defines a value function  $V(p, j, q)$ . Intuitively,  $V(p, j, q)$  is the optimal (minimum) worst-case error for approximating  $\psi$  over the range  $[n(p), n(j_{\max})]$  using at most  $q$  pieces. The first of these pieces is anchored at the left endpoint  $n(p)$ , and the index  $j$  ranges over the candidate right endpoints for this first piece that the recursion considers, with  $p < j \leq j_{\max}$  and  $q \leq k$ , with all points in  $[n(p), n(j)]$  treated as belonging to this same first piece. The function  $V$  is recursively defined as

$$V(p, j, q) = \begin{cases} \text{sPE}(\psi, p, j_{\max}), & \text{if } j \geq j_{\max} \\ \infty, & \text{if } q \leq 0 \\ \min(\max(\text{sPE}(\psi, p, j), & \\ V(j, j+1, q-1)), & \\ V(p, j+1, q)), & \text{otherwise} \end{cases} \quad (2)$$

The function  $\text{sPE} = \text{singlePieceError}(\psi, j_1, j_2)$  measures the error between  $\psi(z)$  and the line joining the points  $(n(j_1), \psi(n(j_1)))$  and  $(n(j_2), \psi(n(j_2)))$ , over the discretized interval  $[n(j_1), n(j_2)]$ . Let  $s_{j_1, j_2}$  be the slope of the line which

equals  $\frac{\psi(n(j_2)) - \psi(n(j_1))}{n(j_2) - n(j_1)}$ .

$$\text{sPE} = \max_{z \in \{n(j_1), \dots, n(j_2)\}} |\psi(z) - \hat{\psi}(z)|, \\ \text{wherein } \hat{\psi}(z) = (\psi(n(j_1)) + (z - n(j_1))s_{j_1, j_2}).$$

The dynamic programming formulation upon memoization runs in time  $O(j_{\max}^3 k)$  assuming that evaluations of  $\psi(z)$  and other arithmetic operations are  $O(1)$ . The optimal error equals the value  $V(0, 1, k)$  and the solution can be recovered in  $O(j_{\max}^2 k)$  steps by traversing the table. Let  $\hat{\psi}$  be the function obtained as a result of running Bellman's algorithm.

b) *Discretization vs. Continuous Error*: We consider the gap between the discretized error  $e(\Delta) = \max_{z \in \{-L, -L+\Delta, -L+2\Delta, \dots, L\}} |\psi(z) - \hat{\psi}(z)|$  and error over the entire continuum of values:  $e(\psi, \hat{\psi}) = \max_{z \in [-L, L]} |\psi(z) - \hat{\psi}(z)|$ . We will derive a correction term that can be used to account for this gap. This correction can be used to modify the singlePieceError calculation used in Eq. (2).

The derivation of this error is detailed in Appendix A and uses the DERIVATIVEINTERVAL procedure that is assumed to exist through Assumption 2. Finally, the existence of this bound provides us a way to control  $\Delta$  to a "small enough" value so that the discretization error can be made to fall below a user-provided limit.

c) *Trade-Off Table*: Finally, we use the dynamic programming formulation to yield a tradeoff table that has the following useful information for a given function  $\psi(z)$ . Let  $k_{\max}$  be an upper limit on the number of breakpoints. The tradeoff table stores, for each value  $k \in [1, k_{\max}]$ , the minimum error  $e(\psi, k)$  obtained by running Bellman's algorithm seeking a function  $\hat{\psi}_k$  with at most  $k$  pieces and information on the optimal breakpoints. Note that we do not really need to run the dynamic programming  $k_{\max}$  times, once for each budget  $k \in [1, k_{\max}]$ . The recursion in Eq. (2) only ever decreases the piece-budget argument  $q$ , by exactly one each time a piece is closed off. Consequently, computing  $V(0, 1, k_{\max})$  by memoization necessarily computes and caches  $V(0, 1, q)$  for every intermediate budget  $q \leq k_{\max}$  along the way. So a single run of Bellman's algorithm with  $k = k_{\max}$  already populates the full memo-table, and we can simply read off the value  $V(0, 1, k)$  for every  $k \in [1, k_{\max}]$  from it, without any additional runs.

## V. OPTIMIZED PWA FOR ENTIRE NETWORK

We will now consider how to approximate the entire network into a PWA function, while ensuring that the overall error  $\max_{\mathbf{x}} |f_N(\mathbf{x}) - \hat{f}_N(\mathbf{x})| \leq \delta$  for a fixed  $\delta$ . Let us suppose we replace each unit in the KAN  $\psi_{i,j}$  by a PWA function  $\hat{\psi}_{i,j}$  incurring an error  $e_{i,j} = \max_{z \in [-L, L]} |\psi(z) - \hat{\psi}(z)|$ . We will derive a bound for the entire NN for the value  $\max_{\mathbf{x} \in X} |f_N(\mathbf{x}) - \hat{f}_N(\mathbf{x})|$ . First, we prove an error propagation bound for each unit  $\psi_{i,j}$  under inputs  $z, \hat{z}$ .



**Lemma 2.** For each unit  $\psi_{i,j}$ , there exists a constant  $\Lambda_{i,j} \geq 0$  such that for all  $z, \hat{z} \in [-L_{i,j}, L_{i,j}]$ ,

$$|\psi_{i,j}(z) - \hat{\psi}_{i,j}(\hat{z})| \leq \mathbf{e}_{i,j} + \Lambda_{i,j}|z - \hat{z}|$$

*Proof.* Let  $\psi = \psi_{i,j}$ ,  $\hat{\psi} = \hat{\psi}_{i,j}$ ,  $\psi'_{\max} = \max_{z \in [-L_{i,j}, L_{i,j}]} \left| \frac{d\psi}{dz} \right|$  and  $\mathbf{e} = \mathbf{e}_{i,j}$ . We may write

$$|\psi(z) - \hat{\psi}(\hat{z})| \leq |\psi(z) - \psi(\hat{z})| + |\psi(\hat{z}) - \hat{\psi}(\hat{z})| \leq \psi'_{\max}|z - \hat{z}| + \mathbf{e}$$

We obtain the result by setting  $\Lambda_{i,j} = \psi'_{\max}$ . Note that the DERIVATIVEINTERVAL (Assumption 2) function can be used to compute a tight upper bound on  $\psi'_{\max}$ .  $\square$

Consider a NN as in Def. 1 with  $K$  layers, input  $\mathbf{x} \in X$  and output  $y$ . Let  $W_1, \dots, W_K$  be the matrices for the hidden layers and  $W_{K+1}$  be the output layer matrix. Also, let  $\psi_{i,j}$ , the  $j^{\text{th}}$  activation function at the  $i^{\text{th}}$  layer be approximated by  $\hat{\psi}_{i,j}$  with the local approximation error satisfying Lemma 2. Consider the PWA approximated network obtained by replacing each  $\psi_{i,j}$  by  $\hat{\psi}_{i,j}$ . Let  $\mathbf{z}_1, \dots, \mathbf{z}_{K+1}$  be the layer inputs, with  $\mathbf{z}_1 = \mathbf{x}$  and  $\mathbf{z}_{i+1} = \sigma_i(W_i \mathbf{z}_i)$  for  $i = 1, \dots, K$ . Similarly, for the PWA approximated network, let  $\hat{\mathbf{z}}_i$  represent the corresponding input to the  $i^{\text{th}}$  hidden layer in the PWA approximated network. Let  $\Lambda_i$  be the vector whose entries correspond to  $\Lambda_{i,j}$  for the activation function  $\psi_{i,j}$  from Lemma 2 and  $\mathbf{e}_i$  be the vector whose entries correspond to the approximation error  $\mathbf{e}_{i,j}$  for the activation function  $\psi_{i,j}$ . Let  $|W_i|$  represents the matrix whose entries are the absolute values of the corresponding entries in  $W_i$  and  $|\mathbf{z}_i - \hat{\mathbf{z}}_i|$  be the vector whose entries consist of the absolute value of entries in  $\mathbf{z}_i - \hat{\mathbf{z}}_i$ .

**Lemma 3.** For  $1 \leq i \leq K$ , we have  $|\mathbf{z}_{i+1} - \hat{\mathbf{z}}_{i+1}| \leq \Lambda_i \odot (|W_i| |\mathbf{z}_i - \hat{\mathbf{z}}_i|) + \mathbf{e}_i$ .

*Proof.* First, we observe that  $|W_i \mathbf{z}_i - W_i \hat{\mathbf{z}}_i| \leq |W_i| |\mathbf{z}_i - \hat{\mathbf{z}}_i|$ . Next, applying Lemma 2, we obtain

$$|\mathbf{z}_{i+1} - \hat{\mathbf{z}}_{i+1}| \leq \Lambda_i \odot (|W_i| |\mathbf{z}_i - \hat{\mathbf{z}}_i|) + \mathbf{e}_i.$$

Recall that for two vectors  $\mathbf{a}, \mathbf{b}$  of the same dimension, we define  $\mathbf{a} \odot \mathbf{b}$  as the vector whose entries are products of the corresponding entries  $a_i b_i$ .  $\square$

We will now state the main result that casts the overall error of each output of the NN in terms of the approximation error  $\mathbf{e}_{i,j}$  at each unit.  $\mathbf{y} = f_N(\mathbf{x})$  and  $\hat{\mathbf{y}} = \hat{f}_N(\mathbf{x})$  for some  $\mathbf{x} \in X$ . Let  $y_l$  and  $\hat{y}_l$  the  $l^{\text{th}}$  component of the output for  $1 \leq l \leq m$ .

**Theorem 1.** For each unit  $\psi_{i,j}$  and for each output index  $1 \leq l \leq m$  there exists a constant  $S_{i,j}^{(l)} \geq 0$  such that the overall approximation error is  $|y_l - \hat{y}_l| \leq \sum_{i=1}^K \sum_{j=1}^{n_i} S_{i,j}^{(l)} \times \mathbf{e}_{i,j}$ . Here  $S_{i,j}^{(l)}$  denotes sensitivity weight constants (distinct from the layer weight matrices  $W_1, \dots, W_{K+1}$  of Def. 1 and the knapsack budget  $W$  of Problem 3).

*Proof.* Note that  $\mathbf{z}_1 = \hat{\mathbf{z}}_1 = \mathbf{x}$  (both networks receive the same input). Therefore,  $|\mathbf{z}_1 - \hat{\mathbf{z}}_1| = 0$ . From Lemma 3, we note that  $|\mathbf{z}_{i+1} - \hat{\mathbf{z}}_{i+1}| \leq \Lambda_i \odot (|W_i| |\mathbf{z}_i - \hat{\mathbf{z}}_i|) + \mathbf{e}_i$ . Expanding on this,

we note that for each entry  $|z_{i+1,r} - \hat{z}_{i+1,r}|$  the bound takes the form

$$|z_{i+1,r} - \hat{z}_{i+1,r}| \leq \sum_{k=1}^i \sum_{j=1}^{n_k} S_{k,j}^{(r)} e_{k,j},$$

for some positive constants  $S_{k,j}^{(r)}$  that bound the contribution of the error incurred in the PWA approximation of  $\psi_{k,j}$  on the  $r^{\text{th}}$  input to the  $(i+1)^{\text{th}}$  hidden layer.

Finally, the output  $y_l$  is obtained as the  $l^{\text{th}}$  component of the vector  $\mathbf{y} = W_{K+1} \mathbf{z}_{K+1}$ . Therefore, we obtain the required bound as a weighted linear combination of approximation error for each activation function.  $\square$

**CK:** This bound is a worst-case surrogate and is not guaranteed to be tight. Most of the slack we see comes from Lemma 2 which uses the Lipschitz constant  $\Lambda_{i,j} = \max_{z \in [-L_{i,j}, L_{i,j}]} |\psi'_{i,j}(z)|$  over each units entire input range rather than the local slope at the inputs actually encountered, and Lemma 3 bounds  $|W_i \mathbf{z}_i - W_i \hat{\mathbf{z}}_i|$  by  $|W_i| |\mathbf{z}_i - \hat{\mathbf{z}}_i|$ , which throws away any cancellation between positive and negative entries, essentially the same sign-agnostic propagation used in interval arithmetic (Section II). The bound is still useful for the knapsack of Section V-A, which only needs the *relative* sensitivity  $S_{i,j}^{(l)}$  across units to be roughly right rather than the absolute error estimate to be tight. Section VII suggests that this is enough in practice.

#### A. Optimized NN Abstraction as Knapsack

Theorem 1 shows how to calculate the error once we have decided upon a PWA abstraction of each unit  $\psi_{i,j}$  yielding the error term  $\mathbf{e}_{i,j}$ . The constants  $S_{i,j}^{(l)}$  corresponding to each unit and output can be computed using a layer-by-layer error propagation analysis that was presented in the proof of the theorem. For the remainder of this section, we will assume that the network has a single output, and we write  $S_{i,j}$  for the corresponding sensitivity weight  $S_{i,j}^{(1)}$ . Alternatively, we may specify a weight vector that combines error bound for each component of the output into an overall weighted error bound to be minimized.

The key problem is therefore to decide how many pieces  $p_{i,j}$  to allocate to each unit  $\psi_{i,j}$ , given a fixed global budget  $B$  on the total number of pieces used across the entire network, i.e.,  $\sum_{i,j} p_{i,j} \leq B$ , so as to minimize the resulting overall approximation error  $\sum_{i,j} S_{i,j} \times \mathbf{e}_{i,j}(p_{i,j})$ . We fix a budget on the total number of pieces, rather than on the error, because it is this number that determines the complexity of the downstream verification problem, especially the MILP formulation that we will provide in Section VI: for a fixed complexity budget  $B$ , we then obtain the allocation of pieces to units that makes the best use of it, i.e., that minimizes the resulting approximation error. We formulate the problem of optimized abstraction as a variant of the well-known knapsack problem called the multi-choice knapsack problem [54].

**Problem 3** (Multi-Choice Knapsack Problem). An instance of the multi-choice knapsack problem is defined as follows:

**Inputs:** We have  $N \geq 1$  options, wherein each option is a set of  $k \geq 1$  items that are available to be chosen and the items belonging to two different options are disjoint. Furthermore, for each option  $i$ , we have a table of non-negative weights (listed in ascending order) and values for the items in option  $i$ :

| Item    | $I_1^{(i)}$   | $I_2^{(i)}$   | $\dots$   | $I_k^{(i)}$ |
|---------|---------------|---------------|-----------|-------------|
| Weights | $w_1^{(i)} <$ | $w_2^{(i)} <$ | $\dots <$ | $w_k^{(i)}$ |
| Value   | $v_1^{(i)}$   | $v_2^{(i)}$   | $\dots$   | $v_k^{(i)}$ |

We are given a total weight budget  $W \geq 0$ . We will assume feasibility: i.e.,  $\sum_{i=1}^N w_1^{(i)} \leq W$ .

**Output:** Choose a set of  $N$  items, choosing precisely one item from each option such that (a) the total weight of chosen items remains within the budget  $W$  and (b) the total value of the chosen items is maximized.

**Theorem 2.** The (decision version) of the multi-choice knapsack problem is **NP-complete**.

Proof is provided in Appendix B. Choosing an optimized abstraction for a NN reduces to a multi-choice knapsack problem: (a) Each option corresponds to a single unit  $\psi_{i,j}$  and thus, there are as many options as the number of units in the KAN; (b) For each option corresponding to the unit  $\psi_{i,j}$ , each item corresponds to a choice of the number of pieces  $p \in [1, k_{\max}]$  for the piecewise linearization  $\hat{\psi}_{i,j}$ , with weight  $w_p^{(i,j)} = p$ , the number of pieces itself, and value  $v_p^{(i,j)} = -S_{i,j} \times e_{i,j}(p)$ , wherein  $e_{i,j}(p)$  is the minimum error achievable using  $p$  pieces for unit  $\psi_{i,j}$ , obtained from the trade-off table discussed in Section IV. Using more pieces for a unit lowers its error  $e_{i,j}(p)$  and hence raises its value  $v_p^{(i,j)}$ , at the cost of a larger weight. (c) The weight budget  $W$  of Problem 3 is the same as the total piece budget  $B$ . Maximizing the total value  $\sum_{i,j} v_p^{(i,j)}$  subject to the weight budget  $B$  is therefore the same problem as minimizing the total weighted error  $\sum_{i,j} S_{i,j} \times e_{i,j}(p_{i,j})$  subject to the piece budget  $B$ , as desired.

Let us assume a multi-choice knapsack problem instance with  $N$  options, each having  $k$  items. The dynamic programming formulation is based on a value function that we will denote  $\text{MO}(i, \hat{W})$  that asks for the optimal (maximum) total value obtainable by selecting items from options  $i$  until  $N$  (inclusive) with a remaining weight budget  $\hat{W}$ .

$$\text{MO}(i, \hat{W}) = \begin{cases} -\infty & \text{if } \hat{W} < 0 \\ 0 & \text{if } i > N \wedge \hat{W} \geq 0 \\ \max_{j=1}^k \left( v_j^{(i)} + \text{MO}(i+1, \hat{W} - w_j^{(i)}) \right) & \end{cases}$$

The optimal answer is obtained by computing  $\text{MO}(1, W)$  through memoization.

**Theorem 3.** The dynamic programming algorithm for multi-choice knapsack runs in time  $O(WNk)$ .

In the setting of optimized NN abstractions, the weights  $w_p^{(i,j)} = p$  are natural numbers by construction, and the weight

budget  $W$  is instantiated as the total piece budget  $B$ . Unlike an error-valued budget, this requires no additional quantization assumption. The running time complexity is linear in the network size, since  $N$  is the number of units in the network.

## VI. MIXED INTEGER LINEAR PROGRAM (MILP) ENCODING

In this section, we will briefly discuss how to solve the range verification problem given a PWA approximation of each activation function  $\psi_{i,j}$  by  $\hat{\psi}_{i,j}$  and corresponding error  $e_{i,j}$ . Our approach is to note that each PWA function  $\psi_{i,j}$  can be written as a finite weighted sum of ReLUs over the domain  $[-L_{i,j}, L_{i,j}]$ . Therefore, any approach that can encode ReLU NNs into a MILP can be directly adapted to encode the PWA abstraction for the purposes of solving the range verification problem. Other verification approaches can be used as well after modifying them to account for the approximation error at each unit. Recall that  $\text{RLu}(x) = \max(x, 0)$ .

**Theorem 4.** Any continuous  $k$ -piece PWA function  $\psi(z)$  (according to Def. 2) with  $k \geq 2$  pieces and  $k-1$  breakpoints can be written as a linear combination of ReLU functions:

$$\psi(z) = \left( y_1 - m_0 \text{RLu}(z_1 - z) + m_1 \text{RLu}(z - z_1) + \sum_{j=2}^{k-1} (m_j - m_{j-1}) \text{RLu}(z - z_j) \right). \quad (3)$$

*Proof.* Let  $g(z)$  represent the RHS of Eq. (3). We verify that  $\psi(z) = g(z)$  by running through the cases  $z \leq z_1$  and then each of the cases  $z \in [z_j, z_{j+1}]$  for  $j \in [1, \dots, k-2]$  and  $z \geq z_{k-1}$ .

First, if  $z \leq z_1$ , verify that  $\psi(z) = y_1 + m_0(z - z_1) = y_1 - m_0 \text{RLu}(z_1 - z)$ . Also,  $\text{RLu}(z - z_j) = 0$  for  $z \leq z_1$  since  $z \leq z_1 < z_j$ .

Suppose  $z \in [z_j, z_{j+1}]$ , then note that  $\text{RLu}(z_1 - z) = 0$  and  $\text{RLu}(z - z_i) = 0$  for all  $i \geq j+1$  and  $\text{RLu}(z - z_i) = (z - z_i)$  whenever  $i \leq j$ . Therefore,

$$\begin{aligned} g(z) - y_1 &= m_1 \text{RLu}(z - z_1) + \sum_{i=2}^j (m_i - m_{i-1}) \text{RLu}(z - z_i) \\ &= m_1(z - z_1) + \sum_{i=2}^j (m_i - m_{i-1})(z - z_i) \\ &= (m_1 + (m_2 - m_1) + \dots + (m_j - m_{j-1}))z \\ &\quad - \underbrace{(m_1(z_2 - z_1) + \dots + m_{j-1}(z_j - z_{j-1}))}_{\Delta_2} - m_j z_j \\ &= m_j z - m_j z_j + y_j - y_1 \end{aligned}$$

This is because  $m_1 + (m_2 - m_1) + \dots + (m_j - m_{j-1}) = m_j$  and furthermore,  $m_1(z_2 - z_1) = y_2 - y_1$ ,  $m_2(z_3 - z_2) = y_3 - y_2$ ,  $\dots$ ,  $m_{j-1}(z_j - z_{j-1}) = y_j - y_{j-1}$ . Therefore,  $\Delta_2 = y_2 - y_1 + y_3 - y_2 + \dots + y_j - y_{j-1} = y_j - y_1$ . Combining,  $g(z) = y_j + m_j(z - z_j) = \psi(z)$  whenever  $z \in [z_j, z_{j+1}]$ . An identical argument applies when  $z \geq z_{k-1}$  to yield  $g(z) = y_{k-1} + m_{k-1}(z - z_{k-1}) = \psi(z)$ . The required identity is thus verified to hold.  $\square$

As a result, the MILP encoding is identical to existing approaches to neural network verification using mixed integer programming [5], [26], [27] with the addition of an extra continuous variable  $\epsilon_{i,j}$  at each node to represent the error

between the PWA approximation and the original result. Due to space limitations, the details of the ReLU encoding are given in Appendix C.

Theorem 4 also points to a more general approach to verification. Because every PWA abstraction can be rewritten as a sum of ReLU units, our abstraction is exactly a ReLU network augmented with a bounded, nondeterministic error term  $\epsilon_{i,j}$  at each unit. Any verification backend that already handles ReLU networks and admits a bounded slack variable per unit can therefore reason over our abstraction directly. MILP-based approaches handle this natively, as detailed in Appendix C. SMT-based tools such as Reluplex [4] and Marabou [24] can do so as well, by treating each  $\epsilon_{i,j}$  as an additional bounded real-valued variable. Bound-propagation approaches such as CROWN [25] and DeepPoly [31] require a further modification to propagate these per-unit error intervals alongside the usual activation bounds. This is not conceptually difficult, but it does require implementation changes rather than a direct as-is application of these tools.

## VII. RESULTS

Our framework applies to any activation function satisfying Assumption 2, which covers B-splines (and hence deep-spline networks), Padé approximations, Chebyshev polynomials, Gaussian processes, and Fourier-series activations, in addition to fixed activations such as tanh, sigmoid, and gelu. We perform our empirical evaluation on KAN networks [10] specifically because each unit in a KAN carries its own independently learned activation function, unlike a standard feedforward network that reuses a single fixed nonlinearity (e.g., tanh or gelu) at every node. KANs use learnable representations at each node, meaning each activation function is non-linear. Crucially, the functions differ across units and thus a single abstraction for the entire network is not feasible. This makes KANs a natural test for our method: every unit demands its own abstraction, and the choice of how to approximate each unit becomes a core decision problem rather than a one-time preprocessing step.

The KANs used in our analysis are trained using the FastKAN library [10]. Specifically, FastKAN replaces the B-spline basis functions of the original KAN with radial basis functions (RBFs), reducing computation time with negligible accuracy loss. Each activation function is a learned weighted sum of Gaussian RBFs, with the weights optimized independently per unit during training so each unit in the network carries a distinct nonlinear activation function. However, these functions satisfy Assumption 2. As a linear combination of basis functions, we can compute bounds on the derivatives of the basis functions as linear combinations of bounds computed over the fixed basis functions. We use an initial interval analysis over the network to fix the domain of each activation function.

The benchmarks are taken from a variety of sources including the KANs used for function approximations in the original paper [10], KANs trained on existing datasets and KAN model that solves differential equations such as the heat equation. In this paper we focus results on four models of various

sizes. Further analyses for the main benchmarks and additional benchmarks are reported in Appendix D, E.

We run experiments on four types of KAN (*Small*, *Medium*, *Medium-2*, and *Large*) networks with increasing number of parameters. Note that number of parameters is an imperfect proxy for verification difficulty: the Large network ([100, 1, 1]) has a single hidden unit, whereas Medium-2 ([2, 10, 2, 1]) is deeper and produces a harder MILP despite having fewer parameters. Figure 3 reflects this: Medium-2 dominates verification time. We then compare the output tightness of our method (*Optimized*) with the baseline (*Vanilla*) approach.

We consider two approaches: (a) **Vanilla**, the baseline method that uses a fixed number of segments  $k$  for each unit and uses Bellman’s algorithm to compute PWA at each node, and (b) **Optimized**, which solves a knapsack problem for total segment budget  $B = k \cdot |\mathcal{S}|$  across splines by solving an ILP, where  $|\mathcal{S}|$  is the total number of splines in the network. **CK:** For the Q2 timing comparison, both approaches use the same total number of pieces: uniformly for *Vanilla*, and via the piece-budget-constrained ILP of Section V-A for *Optimized*. The Q1 tightness comparison instead targets a fixed error budget, as described below.

The *Optimized* pipeline executes four stages for each verification. First, for each univariate activation function  $\psi_{i,j}$ , we run Bellman’s DP to obtain a trade-off curve relating piece count to local error  $e_{i,j}$ . The DP table at each node is computed for the number of segments ranging from 1 until  $\max(S_{\max}, 2k)$ , where  $S_{\max} = 50$  is the default maximum; the  $2k$  term ensures the table extends beyond the uniform budget when  $k$  is large. *Vanilla* simply uses a single fixed  $k$  for all splines. Second, we estimate each activation function’s Lipschitz constant to obtain the sensitivity weights following the approach outlined in the proof of Theorem 1. **CK:** Third, we encode the multi-choice knapsack as an integer program. For the input-width sweep, this program selects one row from each trade-off table to minimize total pieces used subject to a global error budget  $\delta$  (the dual of the formulation in Section V-A, which instead fixes a piece budget and minimizes error). Both directions solve the same trade-off-table DP, just swapping which quantity is the objective and which is the constraint. We fix  $\delta = 1.5 \cdot \delta_{\min}$ , where  $\delta_{\min} = \sum_s \min_k \tilde{e}_s(k)$  is the sum over splines of the minimum achievable Lipschitz-weighted approximation error. The  $1.5\times$  slack gives the allocator room to trade error against pieces rather than pinning it to the best achievable error. Fourth, with the abstraction now fixed, the chosen PWA abstraction is compiled into an MILP with the MIP gap parameter of the solver set to  $\epsilon_{\text{gap}} = 0.15$  and timeout  $T = 500$ . The *Vanilla* approach using the uniform- $k$  abstraction also uses an MILP encoding to solve the verification problem.

In our experiments we consider  $\ell_\infty$ -balls given by  $\mathcal{B}_\infty(\mathbf{x}, r) = \{\mathbf{x}' \mid \|\mathbf{x} - \mathbf{x}'\|_\infty \leq r\}$  with radii  $r \in \{0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0\}$  and we sweep segment counts  $k \in \{1, 5, 10, 20, 30, 40, 50\}$ . Our implementation is based on Python 3.11 and uses Gurobi as the MILP and ILP solver. All experiments are run with a fixed input domain centered at the origin and with fixed random seeds for



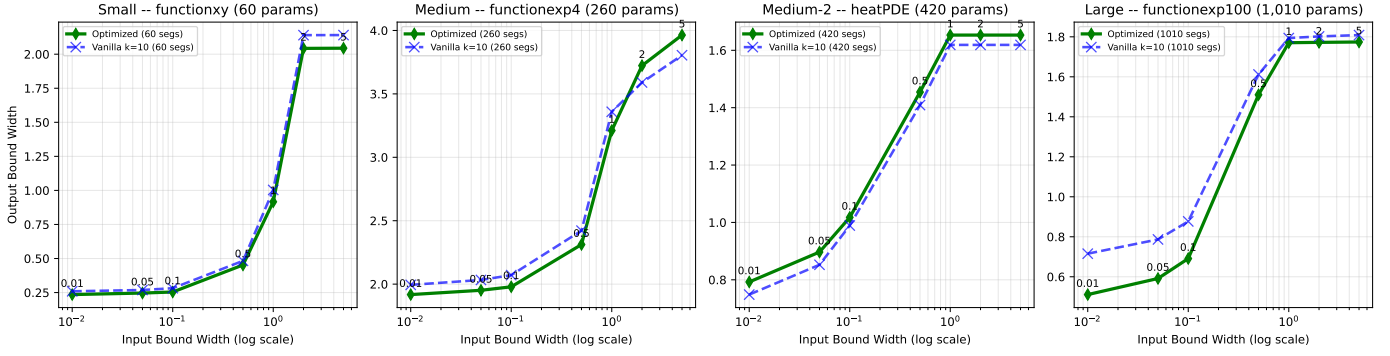


Fig. 2: Tightness of the methods compared by showing how the output bound width changes as input width is varied. We see that across all methods, the output bound width increases as the input bound width increases. **However, our method (*Optimized*) consistently achieves tighter bounds than the baseline (*Vanilla*) across all input widths.** {Small: [2, 2, 1] KAN; Medium: [4, 4, 2, 1] KAN; Medium-2: [2, 10, 2, 1] KAN; Large: [100, 1, 1] KAN; (x params): x trainable parameters per KAN}

reproducibility. **CK:** We report a single run per benchmark rather than averaging over seeds, since the only randomness is in the initial network training and both the DP and ILP allocation steps are deterministic given a trained network. Timing numbers may still vary somewhat from run to run.

**Overarching questions:** We now describe the evaluation method on the benchmarks and ask the following two overarching questions: (1) *does the Optimized approach yield tighter output bounds than the Vanilla baseline at equal segment budgets?* what happens as the segment budget increases? (2) *does our approach scale as model parameters grow?*

**Learning tasks:** We consider four learning tasks: three special function approximation problems and one partial differential equation (PDE) solved using KANs. These examples are taken from the benchmarks of the original paper [11] and consist of four distinct learning scenarios. The tasks are:

- 1) *Small:* funcxy:  $f(x, y) = xy$  (2 inputs; 60 parameters; [2, 2, 1] KAN)
- 2) *Medium:* funcexp4:  $f(x_1, x_2, x_3, x_4) = \exp(\frac{1}{2}(\sin(\pi(x_1^2 - x_2^2)) + \sin(\pi(x_3^2 + x_4^2))))$  (4 inputs; 260 parameters; [4, 4, 2, 1] KAN)
- 3) *Medium-2:* heatPDE:  $u_t = \nu u_{xx}$ ,  $[x, t] \in [0, 1] \times [0, 1]$ ,  $\nu = 0.01$  with Dirichlet boundary and sinusoidal initial conditions. (2 inputs; 420 parameters; [2, 10, 2, 1] KAN)
- 4) *Large:* funcexp100:  $f(x_1, \dots, x_{100}) = \exp(\frac{1}{100} \sum_{i=1}^{100} \sin^2(\frac{\pi x_i}{2}))$  (100 inputs; 1010 parameters; [100, 1, 1] KAN)

The sizes of the KANs are again determined by the best settings as reported in the original paper, and the training is done such that the models achieve a mean squared error (MSE) of less than  $10^{-4}$  on a held-out test set.

**a) Q1: Bound Tightness — Optimized vs. Uniform Allocation:** By varying the input perturbation radius and segment budget, we can evaluate the ablative tightness of the proposed method. Figure 2 evaluates robustness under varying input perturbation radii. The advantage of optimized allocation is most pronounced in the small-radius regime, precisely where tight verification bounds are operationally most valuable. **For**

example, we see that the *Small* benchmark at  $r = 0.01$  shows the optimized abstraction reduces the output width from 0.275 to 0.025, corresponding to an  $11\times$  improvement. The gains are even larger on the *Medium*, *Medium-2*, and *Large* networks, with the *Large* benchmark reaching a  $22\times$  improvement at  $r = 0.05$ . This behavior is consistent with the structure of interval error propagation i.e. for small perturbation sets, approximation error accumulates primarily along a small number of highly sensitive spline paths, allowing the ILP allocator to concentrate segments where they maximally reduce propagated uncertainty, whereas uniform allocation expends budget indiscriminately across low-impact splines.

In contrast, as the perturbation radius increases, the verified bounds produced by both methods gradually tend to converge. In this regime, global nonlinear effects dominate local approximation error. **We also observe that, for the *Large* benchmark at  $r = 0.01$ , the optimized verifier returns  $\infty$ . In this case, the target approximation constraint  $\delta = 1.5 \times \delta_{\min}$  is infeasible under the prescribed segment budget, indicating that there is no admissible PWA abstraction satisfying the target error without increasing  $S_{\max}$ .**

**Noah:** To assess how interval abstraction performs when applied as is, we replace the MILP with standard interval propagation through the uniform  $k = 10$  PWA abstraction. Table I reports the resulting output widths alongside the MILP results of Fig. 2. With small radii, plain interval propagation is  $3\text{--}12\times$  looser than our optimized method, confirming the wrapping-effect coarseness noted in Section II. Since the MILPs are solved to a 0.15 relative optimality gap, their reported bounds can sit marginally above the gap-free interval bounds over the same abstraction, as visible in the *Vanilla* column.

**Noah:** A complementary allocation metric that is independent of the MILP solver: the number of pieces each method needs to certify the same weighted error bound  $\delta = 1.5 \delta_{\min}$ . Across all four benchmarks the optimized allocation needs  $8\text{--}22\%$  fewer pieces than the best uniform choice of  $k$  (Table II).

**b) Q2: Computation Time Comparison:** Figure 3 reports end-to-end verification time as a function of the segment budget.

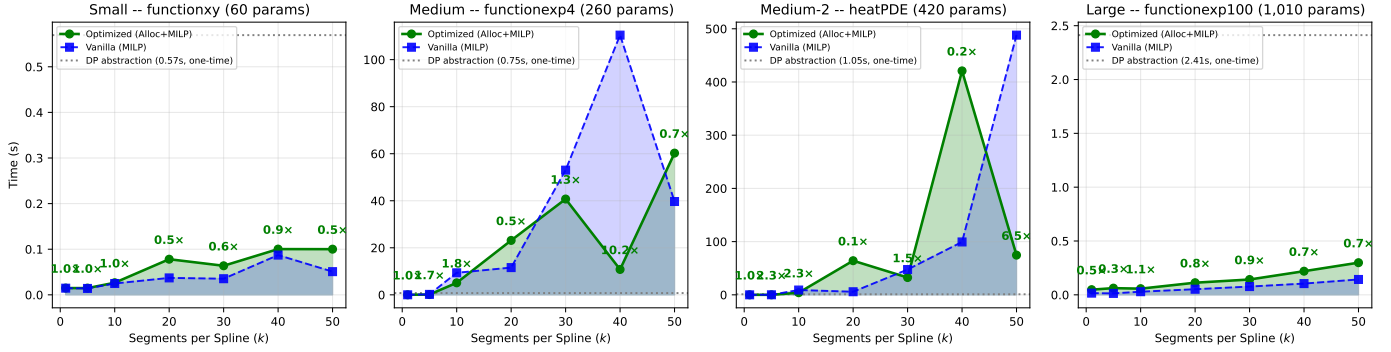


Fig. 3: End-to-end runtime as a function of  $k$ . *Optimized*’s per-query cost is the sum of the one-time DP abstraction (dotted line) and the downstream MILP (green); *Vanilla* incurs only the MILP cost (dashed). On *Medium-2*, where *Vanilla*’s MILP scales sharply with  $k$ , *Optimized* wins end-to-end at moderate-to-high  $k$ . On Small and Large, *Vanilla*’s MILP is already cheap and the DP overhead dominates single-query *Optimized* runtime; the multipliers on the green curve show the MILP-stage runtime ratio (*Vanilla* time / *Optimized* time), which is the relevant quantity when the abstraction is amortized over multiple verification queries on the same network. A ratio above one indicates that *Optimized*’s MILP stage is faster than *Vanilla*’s at that budget; a ratio below one indicates the opposite. {Small: [2, 2, 1] KAN; Medium: [4, 4, 2, 1] KAN; Medium-2: [2, 10, 2, 1] KAN; Large: [100, 1, 1] KAN; (x params): x trainable parameters per KAN}

TABLE I: Noah: Mean verified output bound widths from plain interval bound propagation (IBP) through the uniform  $k=10$  PWA abstraction, compared with the MILP-based *Vanilla* and *Optimized* results of Fig. 2.

| Net      | $r$  | IBP   | Vanilla | Optimized |
|----------|------|-------|---------|-----------|
| Small    | 0.01 | 0.260 | 0.259   | 0.235     |
|          | 0.1  | 0.283 | 0.282   | 0.254     |
| Medium   | 0.01 | 2.081 | 1.994   | 1.918     |
|          | 0.1  | 2.172 | 2.071   | 1.979     |
| Medium-2 | 0.01 | 0.752 | 0.748   | 0.792     |
|          | 0.1  | 1.009 | 0.989   | 1.018     |
| Large    | 0.01 | 0.715 | 0.715   | 0.511     |
|          | 0.1  | 0.877 | 0.877   | 0.691     |

TABLE II: Noah: Pieces required to certify the same weighted error bound  $\delta = 1.5\delta_{\min}$ : optimized allocation versus the smallest uniform  $k$  meeting  $\delta$ .

| Net      | Optimized | Uniform            | Savings |
|----------|-----------|--------------------|---------|
| Small    | 182       | 198 ( $k = 33$ )   | 8%      |
| Medium   | 873       | 1,014 ( $k = 39$ ) | 14%     |
| Medium-2 | 1,177     | 1,512 ( $k = 36$ ) | 22%     |
| Large    | 3,220     | 3,636 ( $k = 36$ ) | 11%     |

The one-time dynamic-programming abstraction overhead is modest: 0.15s, 0.43s, 1.01s, and 1.82s for the networks respectively. This preprocessing phase parallelizes across splines, so its cost grows slowly with model size relative to the MILP stage. Consequently, the abstraction overhead becomes negligible when amortized across multiple verification queries. This is important because a single abstraction of the network can be reused across an unlimited number of verification queries such as varying input radius, target output property, or input feature for sensitivity analysis without re-running the DP. In practice, neural network certification pipelines issue many such queries per model [17], so the upfront abstraction cost is amortized over the entire query set, not charged per query.

For the *Small* benchmark, the optimized verifier achieves a consistent 1.6–3.3 $\times$  MILP runtime improvement over uniform allocation across all budgets. The speedup originates from the sparsity induced by adaptive allocation: low-sensitivity splines receive few breakpoints, substantially reducing the size of the downstream MILP encoding. The effect becomes significantly more pronounced on the *Large* benchmark (100 inputs, 1010 parameters). At  $k = 50$ , the optimized verifier’s MILP solves in 0.09s compared to 1.46s for the uniform baseline, a 15.1 $\times$  reduction in solver time. When the one-time DP abstraction cost of 1.82s is included, this overhead is recovered after approximately two verification queries.

This constitutes the principal scalability result of the framework. Under uniform allocation, the MILP formulation grows proportionally to  $|\mathcal{S}| \times k$ , yielding up to 5000 PWA breakpoints for the *Large* benchmark at  $k = 50$ . Such uniformly dense encodings substantially increase branch-and-bound complexity and weaken solver pruning efficiency. In contrast, the optimized formulation concentrates representational complexity only along verification-critical spline regions. The *Medium* and *Medium-2* benchmarks again exhibit non-monotone behavior, with runtime ratios varying as expected. This suggests that minimizing propagated approximation error alone is insufficient as an allocation objective and motivates future formulations to derive better ways to predict the complexity of MILPs generated from the PWA abstractions.

## VIII. CONCLUSION

We have presented a general framework for the range verification of neural networks with nonlinear and learnable activation functions by treating the choice of piecewise affine abstraction as an optimization problem. We bound network-wide error as a weighted linear combination of local errors and cast the problem of allocating pieces across units as a multi-choice knapsack problem. A key property of the

framework is that the abstraction is computed *once per network* and reused across any number of verification queries. This amortization is precisely what makes the upfront cost of optimized allocation worthwhile in practice, and it matches the structure of real certification pipelines, which issue many queries per model [17].

Our experiments on KANs spanning 60 to 1,010 parameters demonstrate the value of this design. **Optimized allocation produces output bounds up to  $22\times$  tighter than uniform allocation, with the largest gains at small input perturbations where verification is most operationally valuable. On the MILP stage, our approach achieves up to  $15.1\times$  faster solve times than uniform allocation on the largest benchmark, and this speedup is recovered after just two verification queries on the same network. CK: The allocation itself is guided by the surrogate bound of Theorem 1 rather than the true propagated error, and as we discuss in Section V, this bound can be loose. We still see empirical gains because the knapsack only needs the bound’s relative ranking of units to be roughly right, not its absolute value to be tight. Future work will focus on reducing abstraction overhead for smaller networks, for instance via lazy or incremental DP, CK: validating the surrogate directly against a sampling-based estimate of the true error on our benchmarks, and incorporating CK: property- and MILP complexity estimates directly into the allocation objective to further improve end-to-end solver performance.**

## REFERENCES

- [1] C. Liu, T. Arnon, C. Lazarus, C. Strong, C. Barrett, and M. J. Kochenderfer, “Algorithms for verifying deep neural networks,” *Foundations and Trends in Optimization*, vol. 4, no. 3-4, pp. 244–404, 2021. [Online]. Available: <http://dx.doi.org/10.1561/24000000035>
- [2] A. Albarghouthi, *Introduction to Neural Network Verification*. verified-deeplearning.com, 2021, <http://verifieddeeplearning.com>.
- [3] A. Irfan, K. D. Julian, H. Wu, C. Barrett, M. J. Kochenderfer, B. Meng, and J. Lopez, “Towards verification of neural networks for small unmanned aircraft collision avoidance,” in *2020 AIAA/IEEE 39th Digital Avionics Systems Conference (DASC)*, 2020, pp. 1–10.
- [4] G. Katz, C. Barrett, D. Dill, K. Julian, and M. Kochenderfer, “Reluplex: An efficient smt solver for verifying deep neural networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1702.01135v2>
- [5] V. Tjeng, K. Xiao, and R. Tedrake, “Evaluating robustness of neural networks with mixed integer programming,” 2017. [Online]. Available: <https://arxiv.org/abs/1711.07356>
- [6] M. Casadio, E. Komendantskaya, M. L. Daggett, W. Kokke, G. Katz, G. Amir, and I. Refaeli, “Neural network robustness as a verification property: A principled case study,” in *Computer Aided Verification*, S. Shoham and Y. Vizel, Eds. Cham: Springer International Publishing, 2022, pp. 219–231.
- [7] S. R. Dubey, S. K. Singh, and B. B. Chaudhuri, “Activation functions in deep learning: A comprehensive survey and benchmark,” *Neurocomputing*, vol. 503, pp. 92–108, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231222008426>
- [8] P. Bohra, J. Campos, H. Gupta, S. Aziznejad, and M. Unser, “Learning activation functions in deep (spline) neural networks,” *IEEE Open Journal of Signal Processing*, vol. 1, pp. 295–309, 2020.
- [9] S. Aziznejad, H. Gupta, J. Campos, and M. Unser, “Deep neural networks with trainable activations and controlled lipschitz constant,” *IEEE Transactions on Signal Processing*, vol. 68, pp. 4688–4699, 2020.
- [10] Z. Li, “Kolmogorov-arnold networks are radial basis function networks,” *arXiv preprint arXiv:2405.06721*, 2024.
- [11] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljagic, T. Y. Hou, and M. Tegmark, “KAN: Kolmogorov-arnold networks,” in *The Thirtieth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=Ozo7qJ5vZi>
- [12] R. Bellman and R. Roth, “Curve fitting by segmented straight lines,” *J. Amer. Statist. Assoc.*, vol. 64, pp. 1079–1084, 1969.
- [13] R. Bellman, “On the approximation of curves by line segments using dynamic programming,” *Commun. ACM*, vol. 4, no. 6, pp. 284–284, 1961.
- [14] Gurobi Optimization, LLC, “Gurobi Optimizer Reference Manual,” 2024. [Online]. Available: <https://www.gurobi.com>
- [15] IBM ILOG CPLEX Optimization Studio, IBM, 2021, version 20.1.0. [Online]. Available: <https://www.ibm.com/products/ilog-cplex-optimization-studio>
- [16] MOSEK ApS, *MOSEK Optimization Suite*, 2022, version 10.0. [Online]. Available: <https://www.mosek.com/>
- [17] G. Singh, T. Gehr, M. Püschel, and M. Vechev, “An abstract domain for certifying neural networks,” *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 1–30, 2019.
- [18] M. Bojarski, D. Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, “End to end learning for self-driving cars,” 2016. [Online]. Available: <https://arxiv.org/abs/1604.07316>
- [19] S. Gu, E. Holly, T. Lillicrap, and S. Levine, “Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates,” in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 3389–3396.
- [20] G. Clark, J. Campbell, and H. Amor, *Learning Predictive Models for Ergonomic Control of Prosthetic Devices*. [Online]. Available: <https://arxiv.org/pdf/2011.07005>
- [21] I. Goodfellow, J. Shlens, and C. Szegedy, *Published as a conference paper at ICLR 2015 EXPLAINING AND HARNESSING ADVERSARIAL EXAMPLES*, 2015. [Online]. Available: <https://arxiv.org/pdf/1412.6572>
- [22] T. Bai, J. Luo, J. Zhao, B. Wen, and Q. Wang, “Recent advances in adversarial training for adversarial robustness,” *arXiv preprint arXiv:2102.01356*, 2021.
- [23] S. H. Silva and P. Najafirad, “Opportunities and challenges in deep learning adversarial robustness: A survey,” *arXiv preprint arXiv:2007.00753*, 2020.
- [24] G. Katz, D. Huang, D. Ibeling, K. Julian, C. Lazarus, R. Lim, P. Shah, S. Thakoor, H. Wu, A. Zeljić, D. Dill, M. Kochenderfer, and C. Barrett, *The Marabou Framework for Verification and Analysis of Deep Neural Networks*, 07 2019, pp. 443–452.
- [25] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel, “Efficient neural network robustness certification with general activation functions,” ser. NIPS’18. Curran Associates Inc., 2018, pp. 4944–4953.
- [26] S. Dutta, X. Chen, S. Jha, S. Sankaranarayanan, and A. Tiwari, “Sherlock - a tool for verification of neural network feedback systems,” in *Proceedings of the 22nd ACM International Conference on Hybrid Systems: Computation and Control*, ser. HSCC ’19. Association for Computing Machinery, 2019, pp. 262–263. [Online]. Available: <https://doi.org/10.1145/3302504.3313351>
- [27] A. Lomuscio and L. Maganti, “An approach to reachability analysis for feed-forward relu neural networks,” 2017. [Online]. Available: <https://arxiv.org/abs/1706.07351>
- [28] X. Huang, M. Kwiatkowska, S. Wang, and M. Wu, “Safety verification of deep neural networks: A survey,” *Communications of the ACM*, vol. 63, no. 11, pp. 139–150, 2020.
- [29] K. Majd, G. M. Clark, T. Khandait, S. Zhou, S. Sankaranarayanan, G. Fainekos, and H. B. Amor, “Safe robot learning in assistive devices through neural network repair,” in *Conference on Robot Learning, CoRL 2022, 14-18 December 2022, Auckland, New Zealand*, ser. Proceedings of Machine Learning Research, K. Liu, D. Kulic, and J. Ichnowski, Eds., vol. 205. PMLR, 2022, pp. 2148–2158. [Online]. Available: <https://proceedings.mlr.press/v205/majd23a.html>
- [30] T. Kushner, S. Sankaranarayanan, and M. Breton, “Conformance verification for neural network models of glucose-insulin dynamics,” in *Hybrid Systems: Computation and Control (HSCC)*, 2020, pp. 13:1–13:12.
- [31] G. Singh, R. Ganvir, M. Püschel, and M. Vechev, “Beyond the single neuron convex barrier for neural network certification,” in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32. Curran Associates, Inc., 2019. [Online]. Available: [https://proceedings.neurips.cc/paper\\_files/paper/2019/file/0a9fd1bb17feb6ccb7ec405cfb85222c4-Paper.pdf](https://proceedings.neurips.cc/paper_files/paper/2019/file/0a9fd1bb17feb6ccb7ec405cfb85222c4-Paper.pdf)
- [32] A. A. Howard, B. Jacob, S. H. Murphy, A. Heinlein, and P. Stinis, “Finite basis kolmogorov-arnold networks: domain decomposition for data-driven and physics-informed problems,” *arXiv*

- (Cornell University), Jun 2024. [Online]. Available: <https://www.semanticscholar.org/paper/Finite-basis-Kolmogorov-Arnold-networks%3A-domain-for-Howard-Jacob/6be75a69bb14192a26be00ff51c9a2f086f26b41>
- [33] J. D. Toscano, L.-L. Wang, and G. E. Karniadakis, "Kkans: Kurkova-kolmogorov-arnold networks and their learning dynamics," 2024. [Online]. Available: <https://arxiv.org/abs/2412.16738>
  - [34] R. Genet and H. Inzirillo, "Tkan: Temporal kolmogorov-arnold networks," *arXiv preprint arXiv:2405.07344*, 2024.
  - [35] A. Mahara, N. D. Rishe, and L. Deng, "The dawn of kan in image-to-image (i2i) translation: Integrating kolmogorov-arnold networks with gans for unpaired i2i translation," *arXiv preprint arXiv:2408.08216*, 2024.
  - [36] D. W. Abueidda, P. Pantidis, and M. E. Mobasher, "Deepokan: Deep operator network based on kolmogorov arnold networks for mechanics problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 436, p. 117699, 2025.
  - [37] R. Moore, R. Kearfott, and M. Cloud, *Introduction to Interval Analysis*, ser. Other Titles in Applied Mathematics. Cambridge University Press, 2009. [Online]. Available: <https://books.google.com.my/books?id=kd8FmmN7sAoC>
  - [38] L. Pulina and A. Tacchella, "Challenging smt solvers to verify neural networks," *AI Commun.*, vol. 25, no. 2, pp. 117–135, Apr. 2012.
  - [39] R. Ehlers, "Formal verification of piece-wise linear feed-forward neural networks," in *Automated Technology for Verification and Analysis (ATVA)*, ser. Lecture Notes in Computer Science, D. D'Souza and K. N. Kumar, Eds. Springer, 2017, pp. 269–286.
  - [40] H.-D. Tran, D. Manzanar Lopez, P. Musau, X. Yang, L. V. Nguyen, W. Xiang, and T. T. Johnson, "Star-based reachability analysis of deep neural networks," in *Formal Methods (FM)*. Berlin, Heidelberg: Springer-Verlag, 2019, pp. 670–686.
  - [41] Z. Shi, Q. Jin, Z. Kolter, S. Jha, C.-J. Hsieh, and H. Zhang, "Neural network verification with branch-and-bound for general nonlinearities," in *Tools and Algorithms for the Construction and Analysis of Systems. TACAS 2025*, ser. Lecture Notes in Computer Science, A. Gurfinkel and M. Heule, Eds. Cham: Springer, Cham, 2025, vol. 15696.
  - [42] S. Wang, H. Zhang, K. Xu, X. Lin, S. Jana, C.-J. Hsieh, and J. Z. Kolter, "Beta-crown: Efficient bound propagation with per-neuron split constraints for neural network robustness verification," in *Advances in Neural Information Processing Systems 34 (NeurIPS 2021)*, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds. Curran Associates, Inc, 2021, pp. 29 909–29 921.
  - [43] R. Ivanov, J. Weimer, R. Alur, G. J. Pappas, and I. Lee, "Verisig: Verifying safety properties of hybrid systems with neural network controllers," in *Proceedings of the 22nd ACM International Conference on Hybrid Systems: Computation and Control*, ser. HSCC '19. ACM, 2019, pp. 169–178.
  - [44] B. Heiderich, M.-L. Schumacher, and M. Huber, "Training and verifying robust kolmogorov-arnold networks," in *ICLR 2025 Workshop: VerifAI: AI Verification in the Wild*, 2025. [Online]. Available: <https://openreview.net/forum?id=N9hl7CShRA>
  - [45] A. Polo-Molina, D. Alfaya, and J. Portela, "Monokan: Certified monotonic kolmogorov-arnold network," 2024. [Online]. Available: <https://arxiv.org/abs/2409.11078>
  - [46] H. Shen, C. Zeng, J. Wang, and Q. Wang, "Reduced effectiveness of kolmogorov-arnold networks on functions with noise," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.
  - [47] C. Zeng, J. Wang, H. Shen, and Q. Wang, "Kan versus mlp on irregular or noisy functions," *arXiv preprint arXiv:2408.07906*, 2024.
  - [48] S. Goyal, K. Dvijotham, R. Stanforth, R. Bunel, C. Qin, J. Uesato, R. Arandjelovic, T. A. Mann, and P. Kohli, "On the effectiveness of interval bound propagation for training verifiably robust models," *CoRR*, vol. abs/1810.12715, 2018. [Online]. Available: <http://arxiv.org/abs/1810.12715>
  - [49] S. S. Bhattacharjee, "TorchKAN: Simplified KAN model with variations," <https://github.com/lssb/torchkan/>, 2024.
  - [50] A. A. Afzal, "rkan: Rational kolmogorov-arnold networks," 2024. [Online]. Available: <https://arxiv.org/abs/2406.14495>
  - [51] S. SS, K. AR, G. R, and A. KP, "Chebyshev polynomial-based kolmogorov-arnold networks: An efficient architecture for nonlinear function approximation," 2024. [Online]. Available: <https://arxiv.org/abs/2405.07200>
  - [52] A. S. Chen, "Gaussian process kolmogorov-arnold networks," 2024. [Online]. Available: <https://arxiv.org/abs/2407.18397>
  - [53] J. Xu, Z. Chen, J. Li, S. Yang, W. Wang, X. Hu, and E. C. H. Ngai, "Fourierkan-gcf: Fourier kolmogorov-arnold network – an effective and efficient feature transformation for graph collaborative filtering," 2024. [Online]. Available: <https://arxiv.org/abs/2406.01034>
  - [54] H. Kellerer, U. Pferschy, and D. Pisinger, "The multiple-choice knapsack problem," in *Knapsack Problems*. Springer, Berlin, Heidelberg, 2004. [Online]. Available: [https://doi.org/10.1007/978-3-540-24777-7\\_11](https://doi.org/10.1007/978-3-540-24777-7_11)

## APPENDIX A DISCRETIZATION ERROR

**Theorem 5.** Consider any interval  $[n(j_1), n(j_2)]$  for  $0 \leq j_1 < j_2 \leq j_{\max}$  and let  $e_{j_1, j_2}$  denote the error between  $\psi$  and  $\hat{\psi}$  over all the discretization points in the interval  $[n(j_1), n(j_2)]$ . Also, let  $c_{\max}$  be the maximum absolute value of the slope of any piece of  $\hat{\psi}$  over the interval

$$e_{j_1, j_2} \leq \max_{z \in [n(j_1), n(j_2)]} |\psi(z) - \hat{\psi}(z)| \leq e_{j_1, j_2} + (c_{\max} + \psi'_{\max})\Delta$$

wherein  $\psi'_{\max} = \max_{z \in [n(j_1), n(j_2)]} \left| \frac{d\psi}{dz} \right|$ .

We start by proving a useful lemma that derives an error bound for an interval between two consecutive grid points  $[n(j), n(j+1)]$  for  $1 \leq j < j_{\max}$ .

**Lemma 4.** Let  $e_j = |\psi(n(j)) - \hat{\psi}(n(j))|$  for some index  $j \in [1, j_{\max}]$ . For any  $z \in [n(j), n(j+1)]$ , the error  $|\psi(z) - \hat{\psi}(z)| \leq e_j + (\psi'_{\max} + |c|)\Delta$ , wherein  $\psi'_{\max} = \max_{z \in [n(j), n(j+1)]} \left| \frac{d\psi}{dz} \right|$  and  $\hat{\psi}(z) = cz + d$  for  $z \in [n(j), n(j+1)]$

*Proof.* Consider any  $z \in [n(j), n(j+1)]$ . Note that by the mean value theorem:  $\psi(z) = \psi(n(j)) + \psi'(z^*)(z - n(j))$  for some  $z^* \in [n(j), z]$ . Therefore, we have,  $|\psi(z) - \psi(n(j))| \leq |\psi'(z^*)| \times |z - n(j)| \leq \psi'_{\max}\Delta$  which leads us to:

$$\begin{aligned} |\psi(z) - \hat{\psi}(z)| &\leq |\psi(z) - \psi(n(j)) + \psi(n(j)) - \hat{\psi}(n(j)) + \hat{\psi}(n(j)) - \hat{\psi}(z)| \\ &\leq |\psi(z) - \psi(n(j))| + |\psi(n(j)) - \hat{\psi}(n(j))| + |\hat{\psi}(n(j)) - \hat{\psi}(z)| \\ &\leq \psi'_{\max}\Delta + e_j + |c(z - n(j))| \leq e_j + (\psi'_{\max} + |c|)\Delta \quad \square \end{aligned}$$

Note that  $\psi'_{\max}$  can be computed using the function `DERIVATIVEINTERVAL` assumed in Assumption 2. Now, we can use Lemma 4 to derive a correction for `singlePieceError` using the following theorem:

**Theorem 5.** Consider any interval  $[n(j_1), n(j_2)]$  for  $0 \leq j_1 < j_2 \leq j_{\max}$  and let  $e_{j_1, j_2}$  denote the error between  $\psi$  and  $\hat{\psi}$  over all the discretization points in the interval  $[n(j_1), n(j_2)]$ . Also, let  $c_{\max}$  be the maximum absolute value of the slope of any piece of  $\hat{\psi}$  over the interval

$$e_{j_1, j_2} \leq \max_{z \in [n(j_1), n(j_2)]} |\psi(z) - \hat{\psi}(z)| \leq e_{j_1, j_2} + (c_{\max} + \psi'_{\max})\Delta$$

wherein  $\psi'_{\max} = \max_{z \in [n(j_1), n(j_2)]} \left| \frac{d\psi}{dz} \right|$ .

*Proof.* Follows directly from applying Lemma 4 over interval  $[n(j), n(j+1)]$  for  $j_1 \leq j < j_2$ . □

## APPENDIX B PROOF OF THEOREM 2

**Theorem 2.** The (decision version) of the multi-choice knapsack problem is **NP**-complete.

*Proof.* The decision Multi-Choice Knapsack (MCK) problem can be shown to be **NP**-complete by a reduction from the classical 0 – 1 knapsack problem, which is known to be **NP**-complete (which in turn reduced from the subset-sum problem).

Given a candidate solution for the decision version of Problem 3 we can perform the following checks i.e. (a) does the total weight of the selected items within the budget specified  $W$ ? (b) does the total value of all items account to at least the target value  $V$ ?, and (c) is exactly one item is selected from each option? . We can verify all these conditions in  $O(Nk)$  polynomial time, leading us to  $\text{MCK} \in \text{NP}$ .

Now, we can reduce from the decision version of the 0 – 1 knapsack problem, where we can set the problem instance as selecting a set of  $N = m$  items each with weight  $w_j \in \mathbb{N}$ , value  $v_j \in \mathbb{N}$ , a weight bound  $W \in \mathbb{N}$ , and a maximum target value  $V \in \mathbb{N}$ . We need to find out: Is there a subset  $S \subseteq \{1, \dots, m\}$  such that  $\sum_{j \in S} w_j \leq W$  and  $\sum_{j \in S} v_j \geq V$ ?

Given such an instance, we can formulate the problem where, for each item  $i$  in the 0 – 1 knapsack problem, we can define an option  $k$  with two items:

- 1) Item  $I_{i,1}^k$  with weight  $w_{i,1}^k = 0$  and value  $v_{i,1}^k = 0$ , corresponding to not selecting the item  $j$
- 2) Item  $I_{i,2}^k$  with weight  $w_{i,2}^k = w_j$  and value  $v_{i,2}^k = v_j$ , corresponding to selecting item  $j$

If the knapsack problem has a subset  $S$  satisfying the constraints of the MCK problem, then for each  $i$  choosing item  $I_{i,2}^i$  if  $i \in S$  and  $I_{i,1}^i$  otherwise, will lead to a valid MCK solution with total weight  $\leq W$  and value  $\geq V$ . Conversely, any feasible solution that corresponds to a selection of items where each option contributes to either no weight and value or weight and value corresponding to that item selected. In that case, the set of indices  $j$  where  $I_{j,2}^j$  is selected forms a feasible solution to the 0 – 1 knapsack problem.

It is clear that this process of converting the problem is polynomial time in number of inputs  $N$ . Thus, we have shown that 0 – 1 knapsack  $\leq_p$  MCK. □



## APPENDIX C MILP ENCODING

We will recall the MILP encoding of ReLU networks and adapt it to our context where the PWA functions have a fixed error bound. It is convenient to view the network as a directed acyclic graph with three types of nodes:

- 1) Activation function nodes that are PWA functions  $\hat{\psi}_{i,j}$  in the network with an error  $e_{i,j}$ .
- 2) Input nodes corresponding to an input  $x_i$  of the network. These nodes do not have incoming edges.
- 3) Output nodes corresponding to an output  $y_j$  of the network. These nodes do not have outgoing edges.

The edges in the networks are labeled with the weights.

For the encoding, we will have the following decision variables:

- 1) We will introduce a continuous variable  $x_i$  for each input variable in the network and  $y_j$  for each output variable.
- 2) For each activation function node  $n_{i,j}$ , we will introduce an output variable  $z_{i,j}$  along with a set of temporary variables  $\alpha_{i,j}$  representing its input, as well as continuous variables  $t_{i,j}^{(0)}, \dots, t_{i,j}^{(k-1)}$  and binary variables  $w_{i,j}^{(0)}, \dots, w_{i,j}^{(k-1)}$  for each ReLU term in the decomposition from Theorem 4. Additionally, we introduce a variable  $\epsilon_{i,j}$  to model the error.

The *node variable* for an input node is the corresponding input variable, for the output node will be its corresponding output variable and for each activation function node will be the output variable  $z_{i,j}$ .

We will now enumerate the constraints needed in the encoding.

**Input Constraints:** We add constraints of the form  $\ell_i \leq x \leq u_i$  based on the input range to the verification problem.

**Activation Function Constraints:** For each activation function node  $n_{i,j}$ , we first create an expression

$$\alpha_{i,j} = \sum_{(n, n_{i,j}) \in \text{incoming}(n_{i,j})} \text{weight}(n, n_{i,j}) \times \text{var}(n).$$

Here  $\text{incoming}(n)$  for a node  $n$  refers to the incoming edges for that node,  $\text{weight}(n, n_{i,j})$  refers to the edge weight for an edge  $n \rightarrow n_{i,j}$  and  $\text{var}(n)$  is the variable associated with a node  $n$ . This encodes the weighted combination of the values from the connected nodes that is input to the activation function node.

Next, we consider the PWA function  $\hat{\psi}(z)$  written out in terms of a weighted combination of ReLU functions as in Eq. (3) in Theorem 4. Let  $z_1, \dots, z_{k-1}$  be the breakpoints. We will need to encode the following relu constraints:

$$t_{i,j}^{(0)} = \text{RLu}(z_1 - \alpha_{i,j}), t_{i,j}^{(1)} = \text{RLu}(\alpha_{i,j} - z_1), \dots, t_{i,j}^{(k-1)} = \text{RLu}(\alpha_{i,j} - z_{k-1}).$$

Each constraint of the form  $t = \text{RLu}(\alpha - z_l)$  where  $t = t_{i,j}^{(l)}$  and  $\alpha = \alpha_{i,j}$  can be encoded as a series of linear inequalities involving a binary variable  $w = w_{i,j}^{(l)}$  that is recalled below:

$$t \geq \alpha, t \geq 0, t \leq \alpha + L_{i,j}(1 - w), t \leq L_{i,j}w,$$

wherein  $\alpha_{i,j} \in [-L_{i,j}, L_{i,j}]$  are the propagated interval bounds on the inputs.

The key constraint relates the output variable of the neural network to the ReLU outputs and the error following Eq. (3).

$$z_{i,j} = \epsilon_{i,j} + \hat{\psi}(z_1) - m_0 t_{i,j}^{(0)} + m_1 t_{i,j}^{(1)} + \sum_{l=2}^{k-1} (m_l - m_{l-1}) t_{i,j}^{(l)}.$$

We add the constraint  $-e_{i,j} \leq \epsilon_{i,j} \leq e_{i,j}$  to note that the error is bounded.

To compute the range over an output  $y_j$ , we simply maximize the objective function  $y_j$  to find the upper bound and minimize it to find the lower bound.

## APPENDIX D SEGMENTS VS. OUTPUT BOUND WIDTH (MAIN BENCHMARKS)

Figure 4 shows how the verified output bound width varies with the per-spline segment budget  $k$  for the four main benchmarks. At  $k = 1$  both methods coincide by construction, since allocating a single segment per spline eliminates any allocation degrees of freedom. As  $k$  grows, *Optimized* consistently achieves tighter bounds than *Vanilla* at every budget level, with the largest gap at small  $k$  narrowing monotonically as both abstractions approach the true function.

The mechanism is the following. The ILP allocator assigns segments in proportion to each spline’s Lipschitz-weighted approximation error: splines on high-sensitivity paths (those with a large cumulative product of Lipschitz constants from the spline to the output) receive more segments, while low-sensitivity splines receive fewer. *Vanilla*, having no knowledge of path sensitivity, distributes segments uniformly and thereby wastes budget on splines whose approximation errors contribute negligibly to the propagated output bound. As  $k$  increases, even uniform allocation eventually assigns enough segments to all splines that per-spline errors become small and the gap closes. On the *Small* benchmark, the optimized verifier achieves a 13% reduction in output width at  $k = 5$  (2.44 versus 2.74), with the gap contracting below 1% as  $k$  approaches 50.

The *Large* benchmark ([100, 1, 1] KAN,  $f(x_1, \dots, x_{100}) = \exp(\frac{1}{100} \sum_{i=1}^{100} \sin^2(\frac{\pi x_i}{2}))$ ) reaches saturation substantially earlier than the other benchmarks, for two compounding reasons. First, each of the 100 input splines approximates the same smooth univariate function  $\sin^2(\pi x/2)$ , whose bounded curvature means the Bellman DP achieves small per-spline error already at modest  $k$ , leaving little headroom above Vanilla. Second, the network’s symmetric structure implies that all 100 splines have nearly identical Lipschitz constants, so the knapsack allocator receives minimal differentiation signal and degrades toward uniform allocation. This explains the general principle that the benefit of adaptive allocation is largest in networks with *heterogeneous* spline sensitivities, precisely the case in networks such as *Medium-2* that combine diverse layer widths and depths.

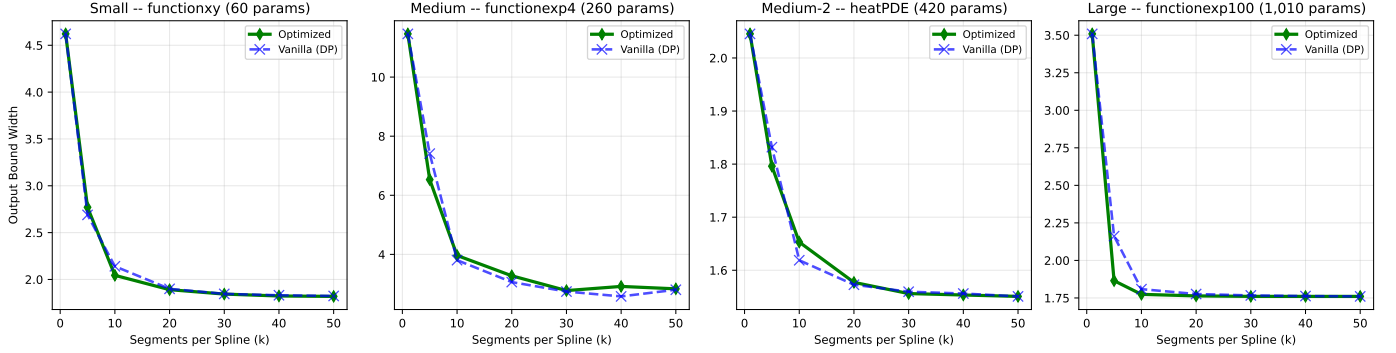


Fig. 4: Output bound width as a function of per-spline segment budget  $k$ . *Optimized* allocates the equivalent global budget  $B = k \cdot |S|$  via the ILP; *Vanilla* assigns  $k$  segments uniformly to every spline. At  $k = 1$  both methods coincide. For larger  $k$ , optimized allocation yields systematically tighter bounds, with the advantage largest at small  $k$  where the ILP can most effectively concentrate segments on high-sensitivity splines, and narrowing as both methods saturate the underlying function. {Small: [2, 2, 1] KAN; Medium: [4, 4, 2, 1] KAN; Medium-2: [2, 10, 2, 1] KAN; Large: [100, 1, 1] KAN; (x params): x trainable parameters per KAN}

## APPENDIX E MORE KAN BENCHMARKS

We evaluate on five additional tasks from the original KAN paper [11], chosen to stress the abstraction framework across a broader range of functional forms than the four main benchmarks. The tasks include a highly oscillatory function (Bessel), functions with rapidly varying curvature near the boundary of their domain (incomplete elliptic integrals), an associated Legendre polynomial evaluated via a wide single-hidden-layer network, and a bivariate special function (spherical harmonics). Together these benchmarks test whether the knapsack allocator’s Lipschitz-weighted sensitivity reasoning transfers to functions whose curvature profiles differ substantially from the smooth exponentials and heat-equation solutions of the main evaluation. The five tasks are:

- 1) *Small*: funcbessel:  $f(x) = \mathcal{J}_0(20x)$  (1 input; 20 parameters; [1, 1] KAN). A highly oscillatory Bessel function of the first kind; the factor of 20 compresses many oscillations into the unit interval, producing large, rapidly alternating curvature that challenges uniform PWA allocation.
- 2) *Small*: funcellipeinc:  $f(x_1, x_2) = E(x_1 | x_2)$ , where  $E(\phi | m) = \int_0^\phi \sqrt{1 - m \sin^2 \vartheta} d\vartheta$  (2 inputs; 70 parameters; [2, 2, 1, 1] KAN). An incomplete elliptic integral of the second kind; the integrand exhibits rapidly varying curvature as  $m \rightarrow 1$ , creating localized high-error regions where adaptive segment concentration is most beneficial.
- 3) *Medium*: funcellipkinc:  $f(x_1, x_2) = K(x_1 | x_2)$ , where  $K(\phi | m) = \int_0^\phi \frac{d\vartheta}{\sqrt{1 - m \sin^2 \vartheta}}$  (2 inputs; 70 parameters; [2, 2, 1, 1] KAN). An incomplete elliptic integral of the first kind; the denominator approaches zero more aggressively than in funcellipeinc, making this a harder approximation target with stronger curvature gradients.
- 4) *Small*: funclegendre:  $f(x) = P_\nu^m(x)$ , where  $P_\nu^m(x) = (1 - x^2)^{m/2} \frac{d^m}{dx^m} P_\nu(x)$  for integer  $m \geq 0$  (1 input; 70 parameters; [1, 100, 1] KAN). The [1, 100, 1] architecture places 100 parallel splines in the single hidden layer, each receiving the same scalar input. Unlike the *Large* benchmark, these splines approximate different nonlinear functions (different rows of the associated Legendre matrix), so their Lipschitz constants vary — giving the knapsack allocator meaningful differentiation signal despite the wide single-layer structure.
- 5) *Small*: funcsphharm:  $f(x_1, x_2) = \Re\{Y_n^m(x_1, x_2)\}$ , where  $Y_n^m(\theta, \phi) = \sqrt{\frac{2n+1}{4\pi} \frac{(n-m)!}{(n+m)!}} P_n^m(\cos \phi) e^{im\theta}$  (2 inputs; 140 parameters; [2, 3, 2, 1] KAN). The real part of a spherical harmonic; the function is smooth globally but exhibits oscillatory angular variation whose sensitivity profile differs across the two input dimensions, providing a bivariate test of cross-path sensitivity estimation.

Figures 5, 6, and 7 report the input-width tightness, segment-budget tightness, and end-to-end timing results for these benchmarks respectively, mirroring the structure of Figures 2, 4, and 3 in the main paper. The trends are consistent: *Optimized* achieves tighter bounds across all input widths and segment budgets, with the largest gains at small perturbation radii and low segment budgets. The oscillatory and rapidly varying benchmarks (Bessel, elliptic integrals) tend to show relatively larger allocation gains, consistent with the expectation that high-curvature splines benefit most from targeted segment concentration.

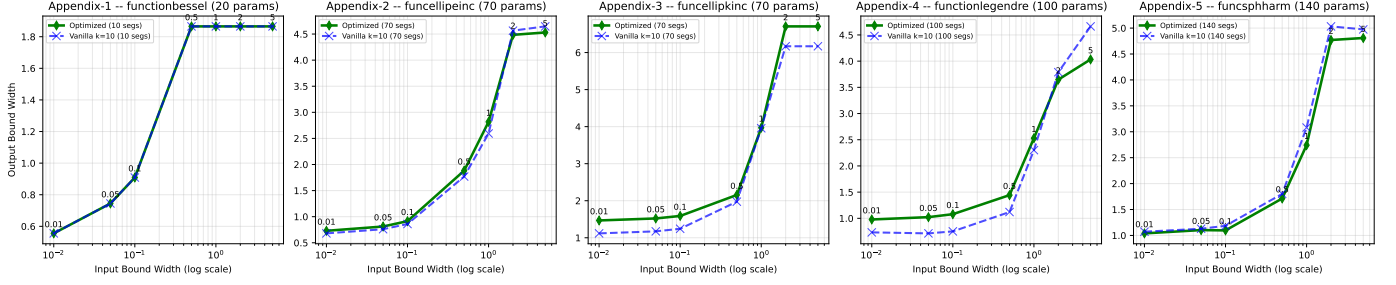


Fig. 5: Output bound width as a function of input perturbation radius  $r$  for the additional benchmarks. The advantage of *Optimized* over *Vanilla* is most pronounced at small  $r$ , where tight bounds are operationally most valuable for robustness certification and the ILP allocator has the most leverage in concentrating segments along high-sensitivity spline paths.

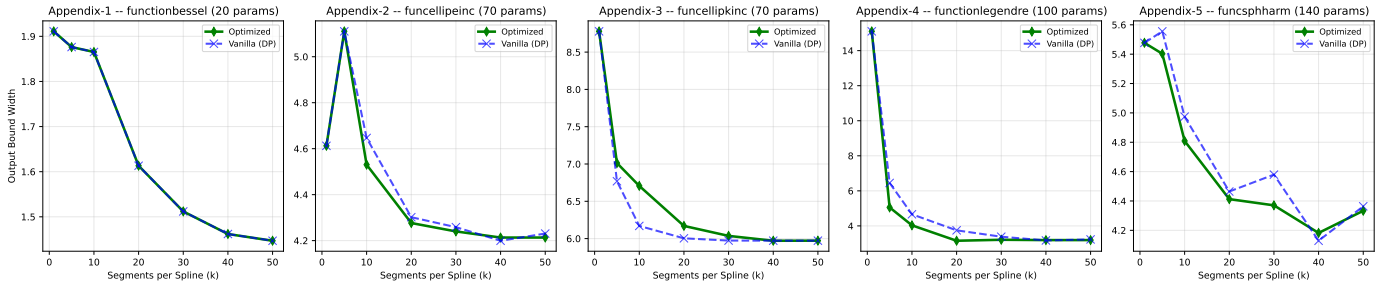


Fig. 6: Output bound width as a function of segment budget  $k$  for the additional benchmarks. *Optimized* consistently achieves tighter bounds than *Vanilla* at every budget level; the gain is largest at low  $k$  where the ILP can most effectively concentrate segments on high-sensitivity splines, and narrows as both methods saturate the underlying function.

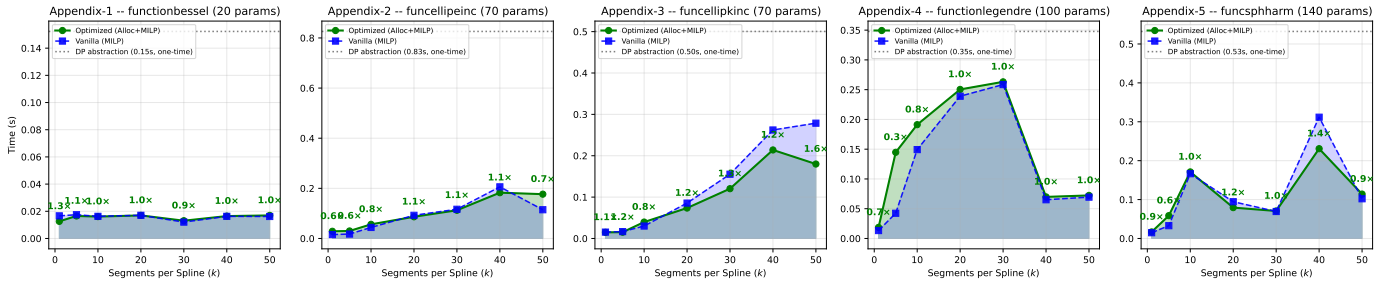


Fig. 7: End-to-end runtime as a function of  $k$  for the additional benchmarks. *Optimized*'s per-query cost is the sum of the one-time DP abstraction (dotted line) and the downstream MILP (green); *Vanilla* incurs only the MILP cost (dashed). The multipliers on the green curve show the MILP-stage speedup, the relevant quantity when the abstraction is amortized across multiple verification queries on the same network.